CHAPITRE 2 : Composants de base (View,Text,Button, Image).

Enseignante : Mme Waderni Sajida

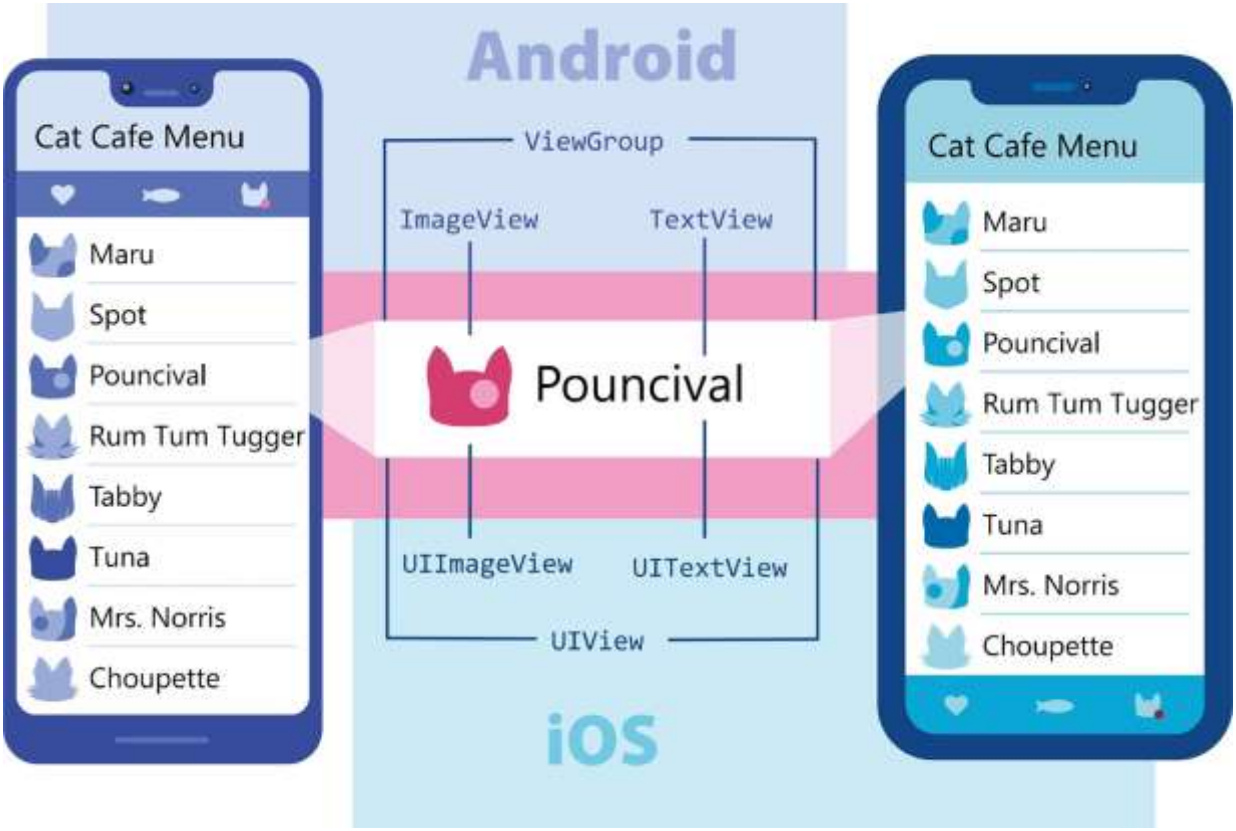
2025/2026

Composants de base et composants natifs

React Native est un framework open source permettant de créer des applications Android et iOS à l'aide de React et des fonctionnalités natives de la plateforme d'applications. Avec React Native, vous utilisez JavaScript pour accéder aux API de votre plateforme ainsi que pour décrire l'apparence et le comportement de votre interface utilisateur à l'aide de composants React : des ensembles de codes réutilisables et imbriqués. Vous pouvez en savoir plus sur React dans la section suivante. Mais d'abord, voyons comment fonctionnent les composants dans React Native.

Views and mobile development

Dans le développement Android et iOS, une **vue** est l'élément de base de l'interface utilisateur : un petit élément rectangulaire sur l'écran qui peut être utilisé pour afficher du texte, des images ou répondre à la saisie de l'utilisateur. Même les plus petits éléments visuels d'une application, comme une ligne de texte ou un bouton, sont des types de vues. Certains types de vues peuvent contenir d'autres vues. Ce sont des vues jusqu'au bout !



Juste un échantillon des nombreuses vues utilisées dans les applications Android et iOS.

Native Components

Dans le développement Android, vous écrivez des vues en Kotlin ou Java ; dans le développement iOS, vous utilisez Swift ou Objective-C. Avec React Native, vous pouvez invoquer ces vues avec JavaScript à l'aide de composants React. Au moment de l'exécution, React Native crée les vues Android et iOS correspondantes pour ces composants. Étant donné que les composants React Native sont soutenus par les mêmes vues qu'Android et iOS, les applications React Native ressemblent à n'importe quelle autre application, ont la même apparence et la même convivialité et fonctionnent comme n'importe quelle autre application. Nous appelons ces composants soutenus par la plateforme **des composants natifs.**

React Native est fourni avec un ensemble de composants natifs essentiels et prêts à l'emploi que vous pouvez utiliser pour commencer à créer votre application dès aujourd'hui. Il s'agit **des composants principaux** de React Native .

React Native vous permet également de créer vos propres composants natifs pour Android et iOS afin de répondre aux besoins uniques de votre application. Nous disposons également d'un écosystème florissant de ces **composants fournis par la communauté.** Consultez Native Directory pour découvrir ce que la communauté a créé.

Core Components

React Native dispose de nombreux composants de base pour tout, des contrôles aux indicateurs d'activité. Vous pouvez les trouver tous documentés dans la section API . Vous travaillerez principalement avec les composants de base suivants :

COMPOSANT D'INTERFACE UTILISATEUR REACT NATIVE	VUE ANDROID	VUE IOS	WEB ANALOGIQUE	DESCRIPTION
`<View>`	`<ViewGroup>`	`<UIView>`	Un non-défilement `<div>`	Un conteneur qui prend en charge la mise en page avec flexbox, le style, certaines gestions tactiles et les contrôles d'accessibilité
`<Text>`	`<TextView>`	`<UITextView>`	`<p>`	Affiche, met en forme et imbrique des chaînes de texte et gère même les événements tactiles
`<Image>`	`<ImageView>`	`<UIImageView>`	`<img>`	Affiche différents types d'images
`<ScrollView>`	`<ScrollView>`	`<UIScrollView>`	`<div>`	Un conteneur de défilement générique pouvant contenir plusieurs composants et vues
`<TextInput>`	`<EditText>`	`<UITextField>`	`<input type="text">`	Permet à l'utilisateur de saisir du texte

Dans la section suivante, vous commencerez à combiner ces composants de base pour découvrir le fonctionnement de React. Jouez avec eux dès maintenant !

Hello World

Expo

```
import React from 'react';
import {View, Text, Image, ScrollView, TextInput} from 'react-native';

const App = () => {
  return (
    <ScrollView>
      <Text>Some text</Text>
      <View>
        <Text>Some more text</Text>
        <Image
          source={{
            uri: 'https://reactnative.dev/docs/assets/p_cat2.png',
          }}
          style={{width: 200, height: 200}}
        />
      </View>
      <TextInput
        style={{
          height: 40,
          borderColor: 'gray',
          borderWidth: 1,
        }}
        defaultValue="You can type in me"
      />
    </ScrollView>
  );
};

export default App;
```

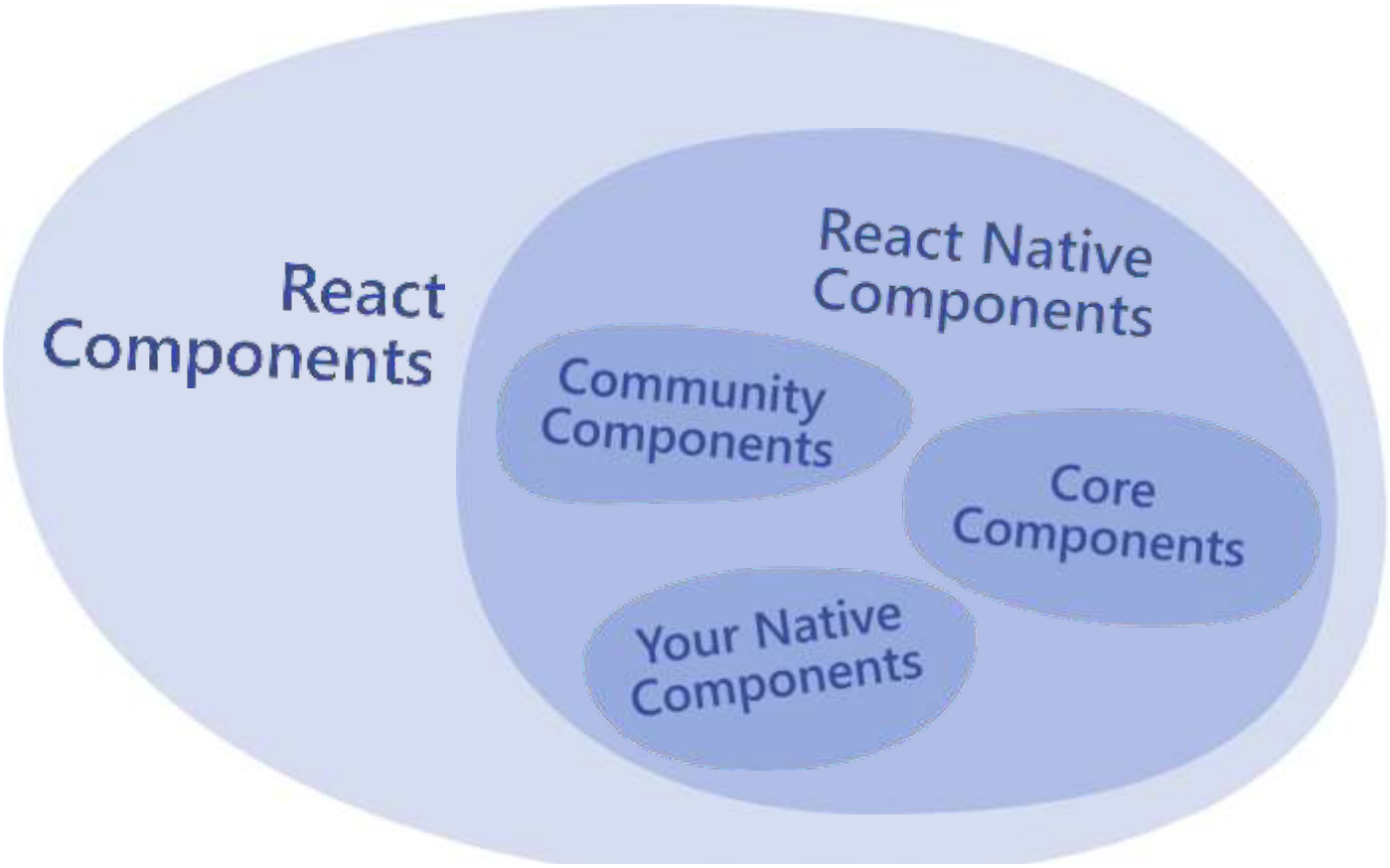
Some text

Some more text



You can type in me

Étant donné que React Native utilise la même structure d'API que les composants React, vous devez comprendre les API des composants React pour commencer. La section suivante constitue une introduction rapide ou un rappel sur le sujet. Cependant, si vous connaissez déjà React, n'hésitez pas à passer à la section suivante .


React
Components
React Native
Components
Community
Components
Core
Components
Your Native
Components

Principes fondamentaux de React

React Native s'exécute sur React , une bibliothèque open source populaire pour la création d'interfaces utilisateur avec JavaScript. Pour tirer le meilleur parti de React Native, il est utile de comprendre React lui-même. Cette section peut vous aider à démarrer ou peut servir de cours de remise à niveau.

Nous allons aborder les concepts de base de React :

- composants
- JSX
- accessoires
- État

Si vous souhaitez approfondir le sujet, nous vous encourageons à consulter la documentation officielle de React .

Your first component

Le reste de cette introduction à React utilise des chats dans ses exemples : des créatures amicales et accessibles qui ont besoin de noms et d'un café pour travailler. Voici votre tout premier composant Cat :

Your Cat

Expo

```
import React from 'react';
import {Text} from 'react-native';

const Cat = () => {
  return <Text>Hello, I am your cat!</Text>;
};

export default Cat;
```

Hello, I am your cat!

Preview | My Device | Android | iOS | Web

Voici comment procéder : pour définir votre `Cat` composant, utilisez d'abord JavaScript `import` pour importer React et `Text` le composant principal de React Native :

```
import React from 'react';
import {Text} from 'react-native';
```

Votre composant démarre comme une fonction :

```
const Cat = () => {};
```

Vous pouvez considérer les composants comme des plans. Tout ce qu'un composant de fonction renvoie est rendu sous forme d' **élément React.** Les éléments React vous permettent de décrire ce que vous voulez voir à l'écran.

Ici, le `Cat` composant rendra un `<Text>` élément :

```
const Cat = () => {
  return <Text>Hello, I am your cat!</Text>;
};
```

Vous pouvez exporter votre composant de fonction avec JavaScript `export default` pour l'utiliser dans toute votre application comme suit :

```
const Cat = () => {
  return <Text>Hello, I am your cat!</Text>;
};

export default Cat;
```

> Il s'agit de l'une des nombreuses façons d'exporter votre composant. Ce type d'exportation fonctionne bien avec Snack Player. Cependant, selon la structure de fichier de votre application, vous devrez peut-être utiliser une convention différente. Cette aide-mémoire pratique sur les importations et exportations JavaScript peut vous aider.

Regardons maintenant de plus près cette `return` déclaration. `<Text>Hello, I am your cat!</Text>` utilise une sorte de syntaxe JavaScript qui rend l'écriture d'éléments pratique : JSX.

JSX

React et React Native utilisent **JSX,** une syntaxe qui vous permet d'écrire des éléments dans JavaScript comme ceci : `<Text>Hello, I am your cat!</Text>`. La documenta React propose un guide complet sur JSX auquel vous pouvez vous référer pour en s

plus. Comme JSX est du JavaScript, vous pouvez utiliser des variables à l'intérieur. Ici, vous déclarez un nom pour le chat, `name`, et vous l'intégrez avec des accolades à l'intérieur de `<Text>`.

Curly Braces

Expo

```
import React from 'react';
import {Text} from 'react-native';

const Cat = () => {
  const name = 'Maru';
  return <Text>Hello, I am {name}!</Text>;
};

export default Cat;
```

Hello, I am Maru!

Preview My Device Android iOS Web

Toute expression JavaScript fonctionnera entre accolades, y compris les appels de fonction comme `{getFullName("Rum", "Tum", "Tugger")}` :

TypeScript JavaScript

Curly Braces

Expo

```
import React from 'react';
import {Text} from 'react-native';

const getFullName = (firstName, secondName, thirdName) => {
  return firstName + ' ' + secondName + ' ' + thirdName;
};

const Cat = () => {
  return <Text>Hello, I am {getFullName('Rum', 'Tum', 'Tugger')}!
</Text>;
};

export default Cat;
```

Hello, I am Rum Tum Tugger!

Preview My Device Android iOS Web

Vous pouvez considérer les accolades comme créant un portail vers les fonctionnalités JS dans votre JSX !

> Parce que JSX est inclus dans la bibliothèque React, cela ne fonctionnera pas si vous ne l'avez pas `import React from 'react'` en haut de votre fichier !

Custom Components

Vous connaissez déjà les composants principaux de React Native . React vous perm d'imbriquer ces composants les uns dans les autres pour créer de nouveaux composants.

Ces composants imbriqués et réutilisables sont au cœur du paradigme React.

Par exemple, vous pouvez imbriquer `Text` et `TextInput` à l'intérieur d'un `View` ci-dessous, et React Native les rendra ensemble :

Custom Components

Expo

```
import React from 'react';
import {Text, TextInput, View} from 'react-native';

const Cat = () => {
  return (
    <View>
      <Text>Hello, I am...</Text>
      <TextInput
        style={{
          height: 40,
          borderColor: 'gray',
          borderWidth: 1,
        }}
        defaultValue="Name me!"
      />
    </View>
  );
};

export default Cat;
```

Hello, I am...

Name me!

Preview My Device Android iOS Web

Developer notes

Androïde **Web**

> Si vous êtes familier avec le développement Web, `<View>` cela `<Text>` peut vous rappe le HTML ! Vous pouvez les considérer comme les balises `<div>` et `<p>` du développem

d'applications.

Vous pouvez restituer ce composant plusieurs fois et à plusieurs endroits sans répéter votre code en utilisant `<Cat>` :

Multiple Components — Expo

```
import React from 'react';
import {Text, View} from 'react-native';

const Cat = () => {
  return (
    <View>
      <Text>I am also a cat!</Text>
    </View>
  );
};

const Cafe = () => {
  return (
    <View>
      <Text>Welcome!</Text>
      <Cat />
      <Cat />
      <Cat />
    </View>
  );
};

export default Cafe;
```

Welcome!
I am also a cat!
I am also a cat!
I am also a cat!

Preview — My Device | Android | iOS | Web

Tout composant qui restitue d'autres composants est un **composant parent.** Ici, `Cafe` il s'agit du composant parent et chacun `Cat` est un **composant enfant.**

Vous pouvez placer autant de chats que vous le souhaitez dans votre café. Chacun `<Cat>` représente un élément unique, que vous pouvez personnaliser avec des acces ;.

Props

Props est l'abréviation de « propriétés ». Les props vous permettent de personnaliser les composants React. Par exemple, ici, vous passez à chacun `<Cat>` un for différent `name` pour `Cat` effectuer le rendu :

TypeScript **JavaScript**

Multiple Props

Expo

```
import React from 'react';
import {Text, View} from 'react-native';

const Cat = props => {
  return (
    <View>
      <Text>Hello, I am {props.name}!</Text>
    </View>
  );
};

const Cafe = () => {
  return (
    <View>
      <Cat name="Maru" />
      <Cat name="Jellylorum" />
      <Cat name="Spot" />
    </View>
  );
};

export default Cafe;
```

Hello, I am Maru!
Hello, I am Jellylorum!
Hello, I am Spot!

Preview My Device Android iOS Web

La plupart des composants principaux de React Native peuvent également être personnalisés avec des props. Par exemple, lorsque vous utilisez `Image`, vous lui transmettez un prop nommé `source` pour définir l'image qu'il affiche :

Props

Expo

```
import React from 'react';
import {Text, View, Image} from 'react-native';

const CatApp = () => {
  return (
    <View>
      <Image
        source={{
          uri:
'https://reactnative.dev/docs/assets/p_cat1.png',
        }}
        style={{width: 200, height: 200}}
      />
      <Text>Hello, I am your cat!</Text>
    </View>
  );
};

export default CatApp;
```



Hello, I am your cat!

Preview My Device Android iOS Web

`Image` possède de nombreux accessoires différents, notamment `style`, qui accepte un objet JS de paires propriété-valeur liées à la conception et à la mise en page.

> Notez les doubles accolades `{{ }}` entourant `style` la largeur et la hauteur de . Dans JSX, les valeurs JavaScript sont référencées avec `{}`. Ceci est pratique si vous transmettez autre chose qu'une chaîne en tant qu'accessoires, comme un tableau ou un nombre : `<Cat food={["fish", "kibble"]} age={2} />`. Cependant, les objets JS sont ***également*** indiqués par des accolades : `{width: 200, height: 200}`. Par conséquent, pour transmettre un objet JS dans JSX, vous devez envelopper l'objet dans **une autre paire** d'accolades : `{{width: 200, height: 200}}`

Vous pouvez construire de nombreuses choses avec des accessoires et les composants principaux `Text` , `Image` , et `View` ! Mais pour construire quelque chose d'interactif, vous aurez besoin d'un état.

State

Bien que vous puissiez considérer les props comme des arguments que vous utilisez pour configurer le rendu des composants, **l'état** est comme le stockage de données personnelles d'un composant. L'état est utile pour gérer les données qui changent au fil du temps ou qui proviennent de l'interaction de l'utilisateur. L'état donne de la mémoire à vos composants !

> En règle générale, utilisez des accessoires pour configurer un composant lors de son rendu. Utilisez state pour suivre toutes les données de composant susceptibles de changer au fil du temps.

L'exemple suivant se déroule dans un café pour chats où deux chats affamés attendent d'être nourris. Leur faim, qui devrait changer au fil du temps (contrairement à leurs noms), est stockée sous forme d'état. Pour nourrir les chats, appuyez sur leurs boutons, ce qui mettra à jour leur état.

Vous pouvez ajouter un état à un composant en appelant le Hook de React `useState` . Un Hook est un type de fonction qui vous permet de vous « connecter » aux fonctionnalités de React. Par exemple, `useState` est un Hook qui vous permet d'ajouter un état aux composants de fonction. Vous pouvez en savoir plus sur les autres types de Hooks dans la documentation de React.

Manuscrit JavaScript

State

Expo

```
import React, {useState} from 'react';
import {Button, Text, View} from 'react-native';

type CatProps = {
  name: string;
};

const Cat = (props: CatProps) => {
  const [isHungry, setIsHungry] = useState(true);

  return (
    <View>
      <Text>
        I am {props.name}, and I am {isHungry ? 'hungry' :
'full'}!
      </Text>
      <Button
        onPress={() => {
          setIsHungry(false);
        }}
        disabled={!isHungry}
        title={isHungry ? 'Give me some food, please!' :
'Thank you!'}
      />
    </View>
  );
};

const Cafe = () => {
  return (
    <>
      <Cat name="Munkustrap" />
```

I am Munkustrap, and I am hungry!

GIVE ME SOME FOOD, PLEASE!

I am Spot, and I am hungry!

GIVE ME SOME FOOD, PLEASE!

Preview | My Device | Android | iOS | Web

Tout d'abord, vous souhaiterez importer `useState` depuis React comme suit :

```
import React, {useState} from 'react';
```

Ensuite, vous déclarez l'état du composant en appelant `useState` sa fonction. Dans cet exemple, `useState` crée une `isHungry` variable d'état :

```
const Cat = (props: CatProps) => {
  const [isHungry, setIsHungry] = useState(true);
  // ...
};
```

Vous pouvez l'utiliser `useState` pour suivre tout type de données : chaînes, nombres, booléens, tableaux, objets. Par exemple, vous pouvez suivre le nombre de fois qu'un chat a été caressé par `const [timesPetted, setTimesPetted] = useState(0)` !

L'appel `useState` fait deux choses :

- il crée une « variable d'état » avec une valeur initiale – dans ce cas, la variable d'état est `isHungry` et sa valeur initiale est `true`
- il crée une fonction pour définir la valeur de cette variable d'état— `setIsHungry`

Peu importe les noms que vous utilisez. Mais il peut être pratique de considérer le modèle comme `[<getter>, <setter>] = useState(<initialValue>)`.

Ensuite, vous ajoutez le `Button` composant principal et lui donnez un `onPress` accessoire :

```
<Button
  onPress={() => {
    setIsHungry(false);
  }}
  //..
/>
```

Maintenant, lorsque quelqu'un appuie sur le bouton, `onPress` il se déclenche, appelant le `setIsHungry(false)`. Cela définit la variable d'état `isHungry` sur `false`. Lorsque `isHungry` est faux, l' accessoire `Button` de 's `disabled` est défini sur `true` et ses `title` modifications sont également les suivantes :

```
<Button
  //..
  disabled={!isHungry}
  title={isHungry ? 'Give me some food, please!' : 'Thank you!'}
/>
```

Vous avez peut-être remarqué que même si `isHungry` est un `const`, il est apparemme réassignable ! Ce qui se passe, c'est que lorsqu'une fonction de définition d'état comr `setIsHungry` est appelée, son composant sera à nouveau rendu. Dans ce cas, la

> `Cat` fonction s'exécutera à nouveau et cette fois, `useState` nous donnera la valeur suivante de `isHungry`.

Enfin, placez vos chats à l'intérieur d'un `Cafe` composant :

```
const Cafe = () => {
  return (
    <>
      <Cat name="Munkustrap" />
      <Cat name="Spot" />
    </>
  );
};
```

> Vous voyez le `<>` et `</>` ci-dessus ? Ces morceaux de JSX sont des fragments. Les éléments JSX adjacents doivent être encapsulés dans une balise englobante. Les fragments vous permettent de le faire sans imbriquer un élément d'encapsulation supplémentaire et inutile comme `View`.

Maintenant que vous avez couvert les composants principaux de React et de React Native, approfondissons certains de ces composants principaux en examinant leur gestion `<TextInput>`

Handling Text Input

Expo

```
import React, {useState} from 'react';
import {Text, TextInput, View} from 'react-native';

const PizzaTranslator = () => {
  const [text, setText] = useState('');
  return (
    <View style={{padding: 10}}>
      <TextInput
        style={{height: 40}}
        placeholder="Type here to translate!"
        onChangeText={newText => setText(newText)}
        defaultValue={text}
      />
      <Text style={{padding: 10, fontSize: 42}}>
        {text
          .split(' ')
          .map(word => word && '🍕')
          .join(' ')}
      </Text>
    </View>
  );
};

export default PizzaTranslator;
```

Type here to translate!

Preview My Device Android iOS Web

Dans cet exemple, nous stockons `text` dans l'état, car il change au fil du temps.

Il existe bien d'autres choses que vous pourriez vouloir faire avec une saisie de texte. Par exemple, vous pouvez valider le texte à l'intérieur pendant que l'utilisateur tape. Pour des exemples plus détaillés, consultez la documentation React sur les composants contrôlés ou la documentation de référence pour TextInput .

La saisie de texte est l'un des moyens par lesquels l'utilisateur interagit avec l'application. Voyons maintenant un autre type de saisie et apprenons à gérer les touches .

Utilisation d'un ScrollView

ScrollView est un conteneur de défilement générique qui peut contenir plusieurs composants et vues. Les éléments défilants peuvent être hétérogènes et vous pouvez faire défiler verticalement et horizontalement (en définissant la propriété `horizontal`).

Cet exemple crée une verticale `ScrollView` avec des images et du texte mélangés.

Using ScrollView

Expo

```
import React from 'react';
import {Image, ScrollView, Text} from 'react-native';

const logo = {
  uri: 'https://reactnative.dev/img/tiny_logo.png',
  width: 64,
  height: 64,
};

const App = () => (
  <ScrollView>
    <Text style={{fontSize: 96}}>Scroll me plz</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{fontSize: 96}}>If you like</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{fontSize: 96}}>Scrolling down</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{fontSize: 96}}>What's the best</Text>
    <Image source={logo} />
    <Image source={logo} />
```

Scroll me plz

Preview My Device Android iOS Web

ScrollViews peut être configuré pour permettre la navigation entre les vues à l'aide de mouvements de balayage à l'aide des `pagingEnabled` accessoires. Le balayage horizontal entre les vues peut également être implémenté sur Android à l'aide du composant ViewPager .

Sur iOS, un ScrollView avec un seul élément peut être utilisé pour permettre à l'utilisateur de zoomer sur le contenu. Configurez les `maximumZoomScale` accessoires `minimumZoomScale` et et votre utilisateur pourra utiliser les gestes de pincement et d'extension pour effectuer un zoom avant et arrière.

Le ScrollView fonctionne mieux pour présenter un petit nombre d'éléments d'une taille limitée. Tous les éléments et vues d'un élément `ScrollView` sont rendus, même s'ils ne sont pas actuellement affichés à l'écran. Si vous avez une longue liste d'éléments qui ne peuvent pas tenir sur l'écran, vous devez utiliser un élément `FlatList` à la place. Découvrons maintenant les vues de liste .

Utilisation des vues de liste

React Native fournit une suite de composants permettant de présenter des listes de données. En général, vous souhaiterez utiliser FlatList ou SectionList .

Le `FlatList` composant affiche une liste déroulante de données changeantes, mais structurées de manière similaire. `FlatList` fonctionne bien pour les longues listes de données, où le nombre d'éléments peut changer au fil du temps. Contrairement au composant plus générique `ScrollView` , le `FlatList` ne rend que les éléments actuellement affichés à l'écran, et non tous les éléments à la fois.

Le `FlatList` composant nécessite deux accessoires : `data` et `renderItem`. `data` est la source d'informations pour la liste. `renderItem` prend un élément de la source et renvoie un composant formaté à restituer.

Cet exemple crée une base `FlatList` de données codées en dur. Chaque élément des `data` accessoires est rendu sous forme de `Text` composant. Le `FlatListBasics` composant restitue ensuite le `FlatList` et tous `Text` les composants.

FlatList Basics

Expo

```
import React from 'react';
import {FlatList, StyleSheet, Text, View} from 'react-
native';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    paddingTop: 22,
  },
  item: {
    padding: 10,
    fontSize: 18,
    height: 44,
  },
});

const FlatListBasics = () => {
  return (
    <View style={styles.container}>
      <FlatList
        data={[
          {key: 'Devin'},
          {key: 'Dan'},
          {key: 'Dominic'},
          {key: 'Jackson'},
          {key: 'James'},
          {key: 'Joel'},
          {key: 'John'},
          {key: 'Jillian'},
          {key: 'Jimmy'},
          {key: 'Julie'},
        ]}
```

Devin

Dan

Dominic

Jackson

James

Joel

John

Jillian

Jimmy

Julie

Preview My Device Android iOS Web

Si vous souhaitez restituer un ensemble de données divisé en sections logiques, peut-être avec des en-têtes de section, similaires à `UITableView` ceux sur iOS, alors une SectionList est la voie à suivre.

SectionList Basics

Expo

```
import React from 'react';
import {SectionList, StyleSheet, Text, View} from 'react-
native';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    paddingTop: 22,
  },
  sectionHeader: {
    paddingTop: 2,
    paddingLeft: 10,
    paddingRight: 10,
    paddingBottom: 2,
    fontSize: 14,
    fontWeight: 'bold',
    backgroundColor: 'rgba(247,247,247,1.0)',
  },
  item: {
    padding: 10,
    fontSize: 18,
    height: 44,
  },
});

const SectionListBasics = () => {
  return (
    <View style={styles.container}>
      <SectionList
        sections={[
          {title: 'D', data: ['Devin', 'Dan', 'Dominic']},
          r
```

D

Devin

Dan

Dominic

J

Jackson

James

Jillian

Jimmy

Joel

John

Julie

Preview | My Device | Android | iOS | Web

L'une des utilisations les plus courantes d'une vue de liste est l'affichage des données que vous récupérez à partir d'un serveur. Pour ce faire, vous devrez en apprendre davantage sur la mise en réseau dans React Native .